

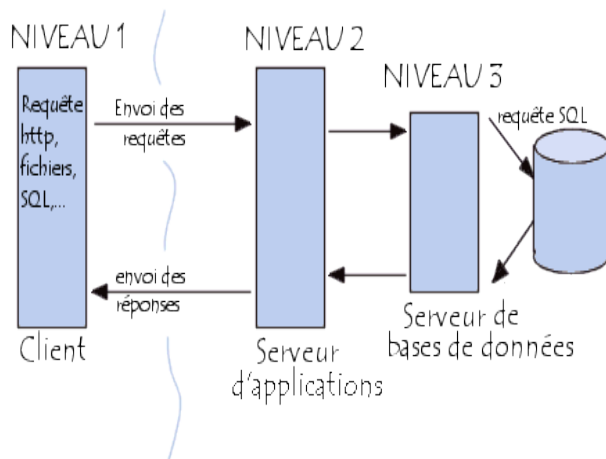
CHAPITRE 1 : LE LANGAGE PHP

PHP (*PHP: Hypertext Preprocessor ou Personal Home Page*) est un langage de scripts spécialement conçu pour le développement d'applications web et pouvant être intégré facilement au HTML.

1. Présentation de l'architecture à 3 niveaux

Dans l'architecture à 3 niveaux (appelée *architecture 3-tier*), il existe un niveau intermédiaire, c'est-à-dire que l'on a généralement une architecture partagée entre :

1. Un client, c'est-à-dire l'ordinateur demandeur de ressources, équipée d'une interface utilisateur (généralement un navigateur web) chargée de la présentation ;
2. Le serveur d'application OU Serveur Web (appelé également **middleware**), chargé de fournir la ressource mais faisant appel à un autre serveur
3. Le serveur de données, fournissant au serveur d'application les données dont il a besoin.



Pour exécuter le script PHP, il faut :

- installer un serveur Web dont **Apache** le plus connu avec les outils comme EasyPHP, LAMP, WAMP, XAMP .
- enregistrer le script de type php dans un sous-répertoire du répertoire **c:\répertoire d'installation\www**

On peut accéder à votre fichier par l'intermédiaire de votre navigateur sur l'adresse :

http://localhost[:port]/ application/fichier.php
ou
http:// adresse IP ou logique du serveur[:port]/application/fichier.php

2 LES BASES DU LANGAGE PHP

Il y a 2 jeux de balises qui servent à délimiter des blocs de code PHP:

Balises PHP : `<?php` `?>`
 Balises de scripts `<script language="php">`
`</script>`

2.1. Les commentaires

Un commentaire vous permet de mettre des lignes dans votre script PHP qui ne seront pas traduites par le navigateur qui les ignorera.. Les commentaires sur votre script qui ne feront qu'une ligne peuvent être amenés comme tel:

//Voici un commentaire

ou

#Voici un commentaire

Si vos commentaires font plus d'une ligne, au lieu de mettre // ou # à chaque début de ligne, vous pouvez mettre directement cela:

/*

Voici un commentaire ne sera analysé par l'interpréteur

***/**

2.2. Les bases de l'écriture dans le code php

PHP fournit des fonctions permettant d'envoyer du texte au navigateur :

- **print(expression)**

- **echo expression.**

- **printf (chaîne formatée);**

Tous les caractères spéciaux, doivent être précédés d'un antislash. Voici un récapitulatif des caractères spéciaux nécessitant l'ajout d'un antislash :

Caractère

\ " guillemet

\ \$ caractère \$

\ / barre oblique inverse (antislash)

\ r retour chariot

\ n retour à la ligne

\ t tabulation

Par exemple, pour marquer Bonjour tout le monde dans un script PHP, il faut mettre:

```

<html>
<head>
<title>Titre</title>
</head>
<body>
<?php
echo "Hello World !";
print ("Bonjour tout le monde");
//Vous pouvez mettre ici un commentaire si vous le souhaitez
print ("Bonsoir tout le monde");
#Pourquoi pas un autre commentaire ??
echo date("d/m/Y")."<br>";
echo date("H:i")."<br>";
?>
</body>
</html>
  
```

Remarque :

1) si on affiche un code HTML de la forme echo "code html", tous les quotes " devront être remplacés par des \", par exemple :

```
echo "< table border='1'> <tr> <td> width =='156'
width =='24'> <p> Contenu du tableau </p>
</td></tr> </table>"
echo "<img scr
='http://www.mon_site.com/mon_image.gif'
width='32' height='32' border='0'>"
```

2) quand l'écho est de la forme echo 'code html', tous les apostrophes ' devront être remplacés par des \', par exemple :

```
echo '< table border=\'1\'> <tr> <td> width ==\'156\'
width ==\'24\'> <p> Contenu du tableau </p> </td></tr>
</table>'
```

2.3. Constantes

- définition de la valeur de la constante :
define("NOM_CONST", valeur);
- vérification d'existence : if(defined(NOM_CONST))...

```
<?php
define("MAIL_MAGALI", "contensin@lim.univ-mrs.fr");
print("envoyez un mail &agrave; ".MAIL_MAGALI);
?>
```

Constantes prédéfinies

PHP_VERSION	version de PHP
PHP_OS	Système sur lequel PHP exécute le script)

```
<?php
echo PHP_VERSION."<BR>";
echo PHP_OS."<BR>";
?>
```

2.3. Les Variables

Les variables PHP peuvent contenir tous types de données : chaînes de caractères, entier, flottant,... PHP n'est pas typé, c'est à dire qu'il n'y a pas de définition de la variable. La seule règle à respecter est que chaque variable doit être précédée de \$.

Une variable peut contenir un caractère comme dans l'exemple qui suit:

Exemple 1 :

```
<?php
$a="ab"; print($a);
$b=12.1; print($b);
$c="<a href='\"/index.php3\">le lien</a>";
print($c);
// Somme de 2 nombres
$a=3; $b=2; $c=$a+$b; print($c);
```

```
// Concatener deux chaînes
$a="a"; $b="b"; $c=$a.$b; print($c);
// opérations d'incrémentations et de décréments
$a=2; $a+=2; $a*=2; print($a++);print("<br>");
print($a);print("<br>");
$a=2; print(--$a);
// on définit la variable $texte et la variable $merci
$texte ="Bonjour tout le monde";
$merci ="merci de visiter ce site";
// on affiche la valeur de $texte (à voir : l'apostrophe et le point)
echo "<b>".$texte."</b><br>";
// on affiche la valeur de $merci
echo "Et ".$merci;
?>
```

Exemple 2 :

```
<?php
$reel = 0.3; $entier = 22;
$chaîne = "bonjour !";
$phrase1 = "bonjour $chaîne!"; $phrase2 = 'bonjour
$chaîne$';

print("un reel : $reel<BR>"); print("un entier :
$entier<BR>");
print("une chaîne : $chaîne<BR>"); print("une phrase :
$phrase1<BR>");
print("du bon usage des guillemets : $phrase2");
?>
```

Exemple 3 :

On peut mélanger le code PHP et le code HTML (le fait qu'il y est beaucoup de HTML ne signifie pas que l'extension .php doit être enlevé !)

```
<html>
<head>
<title>Du PHP dans du HTML</title>
<body>
<font face="Verdana"><b>Bonjour tout le
monde</b></font><br>
<? // rappel : commencement du PHP
// Nous définissons une variable $heure par la fonction
date()
// date() retourne l'heure
// H -> l'heure
// i -> Minute
// \h -> h étant du texte, on rajoute \ devant (antislash)
// de même pour \m\i\n
$heure = date("H \h i \m\i\n");
?>
<font face="Verdana">Il est actuellement <?echo
$heure;?>.</font>
<!-- Ceci affichera "il est actuellement XX h XX min -->
</body>
</html>
```

2.4. Include un fichier dans un script PHP

Il est parfois utile d'inclure des fichiers PHP, HTML ou autres dans un fichier PHP, permettant par exemple de définir plusieurs variables dans un fichier de configuration et de l'inclure dans plusieurs pages de votre script.

Pour inclure un fichier il existe deux fonctions :

include() et require()

Comme avec echo() et print() , la préférence de beaucoup des programmeurs est include() .

Pour inclure un fichier faites :

```
include "mon_fichier.php";
// imaginons que mon_fichier.php contient uniquement
la ligne suivante
// $mon_mail="login@host.com";
// Vous pouvez donc faire appel à la variable
$mon_mail
echo "<a href='mailto:'. $mon_mail. '\">>".
$mon_mail."</a>";
// Affichera login@host.com avec un lien vers ce mail
```

2.5. Tableaux a une dimension

- initialisation trois possibilités équivalentes :

```
1) $tab = array("b","o",10);
2) // définition d'un tableau vide et remplissage
des cases
$tab = array();
$tab[0] = "b"; $tab[1] = "o"; $tab[2] = 10;
$tab[] = "b"; $tab[] = "o"; $tab[] = 10;
```

- accès à un élément d'un tableau de n cases :
\$tab[indice] avec indice dans l'intervalle [0,n-1]
- nombre d'éléments dans un tableau :
\$nb = count(\$tab);

```
<?php
//création d'un tableau simple
$tableau=array(12,15,2,1,8,24,35,5,2);
//accéder à un élément
echo $tableau[0]."<br>";
echo $tableau[1]."<br>";
echo $tableau[3]."<br>";
?>
```

```
<?php
echo "essai avec array : ";
$tab = array("b", "o", "n");
$taille = count($tab);
for($i=0; $i<$taille; $i++)
    echo $tab[$i];

echo "<BR>essai avec [0], [1], ... : ";
$tab2[0] = "h"; $tab2[1] = "e"; $tab2[2] = "l"; $tab2[3]
= "l"; $tab2[4] = "o";
$taille = count($tab2);
for($i=0; $i<$taille; $i++)
    echo $tab2[$i];
```

```
echo "<BR>essai de [] : ";
$tab[] = "j";
$tab[] = "o";
$tab[] = "u";
$tab[] = "r";
$taille = count($tab);
for($i=0; $i<$taille; $i++)
    echo $tab[$i];
?>
```

```
essai avec array : bon
essai avec [0], [1], ... : hello
essai de [] : bonjour
```

Pour définir les **tableaux associatifs**, nous disposons de deux syntaxes :

```
$fiche1 = array();
$fiche1['prenom'] = 'Fabien';
$fiche1['nom'] = 'Torre';
$fiche1['mail'] = 'torre@univ.fr';
$fiche1['url'] = 'http://www.univ.fr/~toto/';
echo 'Mail de '.$fiche1['prenom'].' : '.$fiche1['mail'];
```

```
$fiche2 = array(
    'prenom' => 'Fabien', 'nom' => 'Torre',
    'mail' => 'torre@univ.fr',
    'url' => 'http://www.univ.fr/~toto/',
    'age' => 19,
    'estEtudiant' => true
);
echo 'Mail de '.$fiche2['prenom'].' : '.$fiche2['mail'];
```

Parcours de tableaux PHP : la boucle foreach

Enfin, la boucle *foreach* est dédiée au parcours de tableaux, simples ou associatifs :

```
foreach ($tab as $val) {
    // des instructions ici portant sur $val
}
```

Par exemple,

```
<?php
$tab = array("P", "H", "P", 3);
foreach ($tab as $val) {
    echo "<P> Le tableau contient la valeur $val\n";
}
?>
```

2.6. Les structures de contrôle

L'instruction if

Cette instruction permet au script de s'exécuter seulement si une condition que vous avez défini a été réalisé. Voici un exemple d'utilisation:

```
<?
$humeur="heureux";
if ($humeur=="heureux")
{print "Vous êtes heureux";}
```

```
else
{print "Vous êtes malheureux alors!!";
} ?>
```

Vous pouvez utiliser les opérateurs logiques **&&** (ET) et **||** (OU) dans les conditions

Les boucles

L'instruction for

```
for (compteur; condition; compteur)
{ liste d'instructions }
```

Voici un exemple, pour afficher tous les chiffres de 1 à 10.

```
for ( $i = 1; $i <= 10 ; $i++ )
{ echo $i."<BR>"; }
```

L'instruction while

```
while ( condition )
{
liste d'instructions
}
```

Ce qui donne :

```
$i = 1;
while ( $i <= 10 )
{
echo $i."<br>";
$i++;
}
```

L'instruction switch :

Pour éviter l'imbrication de plusieurs if, switch est alors utilisé, cette instruction est utilisée pour tester plusieurs valeurs d'une seule variable.

```
switch($ce_nombre) {
case 1:
echo $ce_nombre." est égal à 1";
break;
case 2:
echo $ce_nombre." est égal à 2";
break;
case 3:
echo $ce_nombre." est égal à 3";
break;
default:
echo $ce_nombre." est inférieur à 1 ou supérieur à 3";
break;
}
```

2.7. FONCTIONS

```
function
nomfonction($arg1,&$arg2,$arg3="defaultvalue",...,
$argn){
instructions ;
}
```

NB :

- arg1 est passé par valeur ;

- arg2 est passé par adresse : si sa valeur est modifiée dans la fonction, elle est modifiée pour tout le programme ;
- arg3 est optionnel : il peut ne pas être passé lors de l'appel de la fonction puisqu'il a une valeur par défaut. Appel : nomfonction(param1,param2,param3..., paramn);
Pour retourner une valeur : **return**

Exemple 1

```
<?php
// procedure
function sp ($a,$b)
{ $s = $a+$b;$p=$a*$b;
echo " somme=".$s. " produit=" . $p;
}
```

```
// fonction
function somme ($a,$b)
{ $s = $a+$b;
return $s;
}
// appel
$a=2; $b=3;
sp(2,3);
$somme=somme($a,$b);
echo "Somme=" . $somme;
```

```
?>
```

Exemple 2

```
<?php
function test1($a)
{
if ($a)
{ echo "vrai !<br>"; }
else
{ echo "faux !<br>"; }
}
test1(true);
test1(false);
?>
```

Exemple 3 :

```
<?php
function bonjour($a)
{ echo "Bonjour ".$a." !<br>";
echo "<a ref=\"mailto:mon@mail.com\">Contact</a>";
}
bonjour("Manu");
bonjour("Johnny");
?>
```

Exemple 4

```
function form_login_member()
{
echo '<table>';
echo '<form action="liens.php" method="post"
name="login_form">';
echo '<tr>';
```

```

echo '<td>Votre login</td>';
echo '<td><input type="text" name="login" value=""
size="20" maxlength="42"></td>';
echo '</tr>';
echo '<tr>';
echo '<td>Votre password</td>';
echo '<td><input type="password" name="pass"
size="20" maxlength="12"></td>';
echo '</tr>';
echo '<tr>';
echo "<td>";
echo '<input type="submit" name="valider"
value="val">';
echo '<input type="reset" name="annuler"
value="anu">';
echo '</td>';
echo '</tr>';
echo '</form>';
echo '</table>';
}

```

?>

Exemple 5

fonctions.inc.php

```

function form_login_member()
{ ..... }

```

```

<html>
<body>

```

<?php

```
include ("fonctions.inc.php");
```

```
form_login_member();
```

?>

```

</body>
</html>

```

Exemple6

Voici un exemple de programme utilisant une fonction avec un passage par valeur et ensuite par référence :

<?php

```

function modif_tab($tab)
{
    $tab[0] = "j'aime le perl mais je préfère le php.";
}

```

?>

```

<html>
<body>

```

<?php

```
$tab = array("salut", "j'aime le fortran et le cobol");
```

```

// passage de $tab par valeur, la boucle for affichera
// salut, j'aime le fortran et le cobol

```

```

modif_tab($tab);
for ($i = 0; $tab[$i]; $i++)
    echo "$tab[$i] ";

```

```

// passage de $tab par référence, la boucle for affichera
// salut, j'aime le perl mais je préfère le php.

```

```

modif_tab(&$tab);
for ($i = 0; $tab[$i]; $i++)
    echo $tab[$i] ;

```

?>

Exemple 7 : Fonctions récursives

Prenons un exemple simple pour expliquer ce principe à travers le calcul de la puissance d'un nombre entier.

```

<html>
<body>
<?php

```

```

function puissance ($nbr, $puissance)
{
    for ($total = $nbr; $puissance > 1; $puissance--)
        $total = $total * $nbr;
    return ($total);
}

```

```

<html>
<body>

```

<?php

```

$nbr = 2;
$puissance = 4;
$result = puissance($nbr, $puissance);
echo $result;

```

?>

```

</body>
</html>

```

```

function recurse_puissance($nbr, $puissance, $total)
{
    if ($puissance > 1)
        return (recurse_puissance($nbr, $puissance - 1, $total * $nbr));
    return ($total);
}

```

// en une seule ligne :

```
function recurse_puissance($nbr, $puissance, $total)
{
    return (($puissance > 1) ? recurse_puissance($nbr, $puissances - 1, $total * $nbr) : $total);
}
```

```
// APPELS
$nbr = 2;
$puissance = 4;
$result = puissance($nbr, $puissance);
echo $result;
```

```
$result = recurse_puissance(2, 4, 2);
echo $result;
?>
```

```
<?php
function recursion($a)
{
    if ($a < 20) {
        echo "$a\n";
        recursion($a + 1);
    }
}
```

```
recursion(3);
?>
```

```
</body>
</html>
```

Portée des variables globales et locales

Une variable globale n'est pas visible dans une fonction, pour qu'elle le soit il faut le préciser avec global.

```
<?php
// fonction Sport1 $sport est une variable locale
function Sport1(){
    $sport = "volley";
    print ("aimez-vous le $sport ?<BR>");
}
```

```
// fonction Sport2 $sport est une variable globale
function Sport2(){
    global $sport;
    $sport = "badmington";
    print ("aimez-vous le $sport ?<BR>");
}
```

```
// corps du programme
$sport = "ski";
print ("aimez-vous le $sport ?<BR>");
Sport1();
print ("aimez-vous le $sport ?<BR>");
Sport2();
print ("aimez-vous le $sport ?<BR>");
?>
```

aimez-vous le ski ?
 aimez-vous le volley ?

aimez-vous le ski ?
 aimez-vous le badmington ?
 aimez-vous le badmington ?

a. L'instruction GLOBAL

Avant tout, vous devez savoir qu'une fonction qui se trouve à l'extérieur n'est pas directement accessible à l'intérieur d'une fonction.

Exemple Faux :

```
// on définit une variable
$texte="salut";
function salut(){
    // on veut l'afficher
    echo $texte;
}
// on veut afficher "salut"
salut();
```

Il faut donc utiliser la déclaration **GLOBAL** pour permettre de récupérer la valeur \$texte dans l'exemple.

Exemple Bon :

```
// on définit une variable
$texte="salut";
function salut(){
    // on récupère $texte à l'aide de la déclaration GLOBAL
    GLOBAL $texte;
    // on veut l'afficher
    echo $texte;
}
// on veut afficher "salut"
salut();
```

b. L'instruction STATIC

Une variable locale peut être mémorisée d'un appel à l'autre de la fonction en lui mettant static. L'instruction STATIC est utilisée pour garder une trace de la valeur d'une variable entre deux appels de fonction.

Exemple :

```
function titre ($titre) {
    STATIC $nbrtitre=0;
    $nbrtitre++;
    echo "<b>". $nbrtitre. "<b> ". $titre. "<br>";
}
titre("Introduction");
echo "Ce jeu est intéressant au niveau du graphisme<br>";
titre("Histoire");
echo "L'histoire est très original<br>";
// Affichera
// 1 Introduction
// Ce jeu est intéressant au niveau du graphisme
// 2 Histoire
// L'histoire est très original
```

Exemple4 :

```
// Déclaration de la fonction "champ" avec argument
function champ ($nom,$value)
{
    echo "<input type='text' name='\". $nom. \"'
    size='40' value='\". $value. \"'>";
}
// appel de la fonction
champ("sexe","masculin");
// Affichera un champ avec comme nom "sexe" et
comme valeur par défaut "masculin"
```

Exemple5 avec argument par défaut :

Il peut arriver que vous voulez entrer une valeur par défaut au cas où celle-ci n'est pas renseigné :

```
// Déclaration de la fonction "form" avec argument
function form ($mail="Votre mail",$passe="Votre mot de passe")
{
    echo "<input type='text' name='mail' value='\". $mail. \"'>";
    echo "<input type='text' name='passe' value='\". $passe. \"'>";
    if($mail=="Votre mail" AND $passe=="Votre mot de passe"){
        echo "<input type='submit' value='Inscription'>";
    }else{
        echo "<input type='submit' value='Identification'>";
    }
}
// appel de la fonction
form($mail_user,$passe_user);
// Ici nous vérifions par exemple avec les cookies si la
personne est inscrite ou non
// Si oui, l'email et le passe son renseigné
automatiquement avec un bouton Identification
// Si non, il demande l'email et un passe avec un
bouton Inscription
Il suffit donc pour définir une valeur par défaut de faire
comme une variable habituelle ($nom="valeur") . Si
$mail_user est vide, $mail prendra automatiquement la
valeur de $email et inversement. De même pour
$passe ...
```

2.8. CLASSES

Une classe est une collection de variables et de fonctions.

Définition d'une classe

```
class nomclasse{
    var $nomvar1;
    ...
    var $nomvarn;

    // Constructeur de classe
    function nomclasse($par1,$par2, ..... )
    {
        // Initialisation d'une variable
        $this->nomvar1 = $var1;
        $this->nomvar1 = $var2;
```

```
// etc
```

```
}

function nomfonc1(){
    instructions;
    return x;
}
...
function nomfoncn($arg1, $arg2){
    instructions;
}
}
```

Création d'une instance de la classe

Une classe est un type, pour créer un objet (une variable de ce type) il faut utiliser new.
\$nomobjet = new nomclasse;

Accès aux propriétés

\$nomobjet->nomvar1;
\$nomobjet->nomvarn;
Attention : \$nomobjet->\$nomvar1; est faux

Méthodes

\$nomobjet->nomfonc1();
\$nomobjet->nomfoncn("bonjour","au revoir");

Exemple de classe

Voici un exemple de classe permettant l'envoi de courriels.

```
<?php
// Classe permettant l'envoi de courriels
class courriel {
    // Déclaration des attributs de la classe
    var $VarExpediteur = 'bob@monsite.com';
    var $VarDestinataire;
    var $VarSujet;
    var $VarMessage;
    // Constructeur de classe
    function courriel() {
        // Initialisation d'une variable
        $this->VarSujet = '(Aucun sujet)';
    }
    // Méthodes diverses
    function sujet($sujet) {
        $this->varSujet = $sujet;
    }
    function destinataire($destinataire) {
        $this->varDestinataire = $destinataire;
    }
    function expediteur($expediteur) {
        $this->varExpediteur = $expediteur;
    }
    function message($message) {
        $this->varMessage = $message;
    }
    function envoyer() {
        // Vérification du succès de l'envoi
```

```

if( !@mail($this->varDestinataire, // Destinataire
    $this->varSujet, // Sujet
    $this->varMessage, // Message
    'From: ' . $this->varExpéditeur // Autres en-têtes
)) {
    echo 'Erreur lors de l'envoi du courriel!';
}
}

// Instanciation de la classe
$email = new courriel();
// Attribution des valeurs grâce aux méthodes
$email->expéditeur('romeo@monsie.com');
$email->destinataire('juliette@monsie.com');
$email->sujet('Bonjour!');
$email->message('J'ai hâte de te revoir!');
// Envoi du message grâce à la méthode
$email->envoyer();
?>

```

3. LES FORMULAIRES

3.1. Récupérer les informations d'un formulaire par la méthode POST:

La méthode POST est celle qui le plus souvent est utilisée par les sites professionnels, et pour cause, les valeurs des champs d'un formulaire sont secrètement gardés à l'intérieur du script, contrairement à la méthode GET qui transmet les valeurs des champs par l'URL.

Exemple 1 :

Le premier script n'est qu'une page HTML nommée "form_saisie.html".

```

<html>
<head>
<title>Saisie d'informations dans un formulaire</title>
</head>
<body>
<form action="resultat_form.php3" method="POST">
<p>Nom : <input type="text" name="nom">
<br>Prénom : <input type="text" name="prenom">
<br>Adresse : <input type="text" name="adresse">
<br>Code Postale : <input type="text" name="CP">
<br>Ville : <input type="text" name="ville">
<br>Pays : <input type="text" name="pays">
<p><input type="submit" name="soumettre">
<input type="reset">
</form>
</body>
</html>

```

Voyons maintenant le script qui va gérer les données et créer ainsi une page affichant les données (ce script se nomme : "resultat_form.php3") :

```

<html>
<head>
<title>Résultat du formulaire</title>

```

```

</head>
<body>
<h2>Voici le résultat de la soumission du
formulaire</h2>
<? //balise qui indique au serveur Web que vous allez
écrire du code PHP par la suite
print("<p>Nom : <b>$nom</b>\n");
print("<br>Prénom : <b>$prenom</b>\n");
print("<br>Adresse : <b>$adresse</b>\n");
print("<br>Code Postale : <b>$CP</b>\n");
print("<br>Ville : <b>$ville</b>\n");
print("<br>Pays : <b>$pays</b>\n");
?> //balise qui indique au serveur Web de retraiter la
page en HTML simple
</body>
</html>

```

Voyons un autre script qui récupère les données dans un tableau

```

<HTML> <HEAD>
<TITLE>HTML avec PHP</TITLE></HEAD>
<BODY bgcolor='white'>

```

J'ai reçu les données suivantes :

```

<TABLE BORDER=3>
<?php
echo "<TR> <TD>Titre : <TD> $nom </TR>\n";
echo "<TR> <TD>Année : <TD> $prenom </TR>\n";
echo "<TR> <TD>Genre : <TD> $adresse </TR>\n";
echo "<TR> <TD>Pays : <TD> $pays </TR>\n";
echo "<TR> <TD>Réalisateur : <TD> $CP</TR>\n";
echo "<TR> <TD>Résumé : <TD> $ville</TR>\n";
?>
</TABLE></BODY></HTML>

```

Avec les nouvelles version de PHP, vous pourrez rencontrer un problème en récupérant la variable nom simplement en utilisant \$nom, vous devrez la récupérer de cette manière \$_POST[nom] ou \$HTTP_POST_VARS[nom]

Form1.php : récupère les données d'un formulaire simple

```

<html>
<head>
<title>Formulaire simple</title>
</head>
<body>
<?
/* Récupère les entrées saisies dans le formulaire
d'entrée*/
$nom = $HTTP_POST_VARS["Nom"];
$prenom = $HTTP_POST_VARS["Prenom"];
$email = $HTTP_POST_VARS["Email"];
/* Affiche les valeurs des entrées saisies */
echo "$Nom";
echo "$Prenom";
echo "$Email";
?>
</body>
</html>

```

3.2. Verification des donnees saisies par l'utilisateur

Exemple 1 : Un exemple simple et concret :

//le '?' indique que les variables du formulaire seront gérées par le fichier "en cours"

```
<form action=""? method="POST">
<p>Votre Email : <input type="text" name="email">
<input type="submit" name="soumettre">
</form>
```

Vérifier les données saisies par l'utilisateur

Le script utilise trois fonctions :

Première fonction : `afficher_form();` //affiche le formulaire

Cette fonction va être utile si l'utilisateur a oublié de remplir un des champs, il pourra pallier à ce manque sans toutefois devoir remplir tous les autres champs !

Deuxième fonction :
`verification_form();` //verifie les données saisies par l'utilisateur

Cette fonction émet un avertissement si un des champ est mal remplis puis invoque la première fonction afin de corriger la ou les erreurs.

Voici maintenant un exemple de vérification des informations :

Nom du Script : "form.php3"

Le champ prenom est facultatif.

```
<html>
<head>
<title>Vérification des informations saisies</title>
</head>
<body>
<?
// Déclaration des 2 fonctions vu ci haut (cf cours sur les fonctions)
function afficher_form($prenom, $nom, $email) {
?>
<form action=""? method="post">
<table cellpadding="0" cellspacing="0" border="0">
<tr>
<td>Prénom :</td>
<td><input type="text" name="prenom" value="<? echo $prenom ?>"></td>
</tr>
<tr>
<td>Nom :</td>
<td><input type="text" name="nom" value="<? echo $nom ?>"></td>
</tr>
<tr>
<td>Email :</td>
<td><input type="text" name="email" value="<? echo
```

```
$email ?>"></td>
</tr>
<tr>
<td><input type="submit" name="envoyer"
value="Soumettre !">
<input type="Reset">
</td>
</tr>
</table>
</form>
<?
} //fin de declaration de la 1ere fonction
function verification_form($prenom, $nom, $email) {
// verifie si tous les champs obligatoire sont remplis
if(!$nom || !$email) {
echo "<font color='ff0000'>Vous avez omis de remplir
un champ obligatoire !</font>";
if(!$nom) echo "Indiquez votre <b>Nom</b>.<br>";
if(!$email) echo "Indiquez votre <b>Email</b>.<br>";
afficher_form($prenom, $nom, $email);
} //fin du 1er if
else {
echo "<b>Merci !</b>";
}
}

// Début du programme principal
if(!$envoyer) { //si l'utilisateur n'a pa encore appuyé sur
envoyer
echo "<h3>Veuillez saisir les informations</h3>";
afficher_form("", "", "");
}
else { //si le formulaire à été envoyé
verification_form($prenom, $nom, $email);
}
?>
</body>
</html>
```

Les informations sont ici vérifiées mais ne sont stockées que pendant le temps de l'ouverture de la page, à vous ensuite de stocker ces informations dans une Base De Données ou dans des cookies afin de pouvoir les réafficher et s'en servir pendant un passage postérieur de l'utilisateur !

Exemple 2 : Vérification des données saisies par l'utilisateur

Vérifier les infos transmises par le formulaire

*Vérifier que les champs ont été remplis

Pour vérifier qu'un champ du formulaire a bien été remplis, il vous suffit de mettre ce code au début de la page où vous voulez valider votre formulaire:

```
<?
if (empty($nom_du_champ))
{print "Vous n'avez pas rempli tous les champs!!";
break; }
else
?>
```

En fait, il vous suffit de mettre ce code pour chaque champ à vérifier.

***Vérifier que l'email a la bonne forme**

Mettez ce code au début de la page de validation pour vérifier que l'email comporte un point (.) et un arobase (@).

```
<?
$point = strpos($email, ".");
$aroba = strpos($email, "@");
if($point=="")
{echo "Votre email doit comporter un
<b>point</b>";
break;
}
elseif($aroba=="")
{ echo "Votre email doit comporter un <b>'@ '</b>";
break;
}
else
?>
```

Il existe bien sûr d'autres fonctions pour vérifier les champs mais après, cela dépend de votre formulaire mais avec ces 2 fonctions, vous avez la possibilité de déjà bien surveiller un formulaire.

3.3. Récupération d'information à travers une url

1. Présentation d'une URL et ses diverses facettes

Observons une URL :

http://www.truc.com/resultat_form.php3?
prenom=Paul&nom=Martin&email=...

Dans cette URL, tous les éléments situés après le point d'interrogation sont des noms de variables et des valeurs généralement envoyés d'un formulaire au serveur du site.

Chaque élément est composé comme suit : Un nom de variable (prenom, nom, email) et une valeur qui lui est attribué (Paul, Martin, ...). Chaque paire nom/valeur est séparée par une 'Esperluette' ou 'Et Commerciale' (&). Cette forme d'URL se rencontre par exemple après l'envoi d'un formulaire par la méthode GET. Les formulaires qui utilisent la méthode POST n'affichent pas les noms des variables et leurs valeurs dans l'URL.

2. Utilisation des paires nom/valeur

La possibilité de transmettre des informations par l'URL présente un avantage certains pour les développeurs PHP (ou vous même), car chaque paire nom/valeur envoyée ainsi est prêt à être utilisée dans les scripts PHP.

Le script qui lit l'URL va créer automatiquement les variables et leur attribuer une valeur en fonction de la valeur affichée dans l'URL.

Exemple 1 d'utilisation dans un script :

Soit : [http://www.truc.com/info.php3?
nom=Paul&age=18](http://www.truc.com/info.php3?nom=Paul&age=18)

l'URL affichée pendant l'exécution d'un script.

Nom Script : info.php3 //comme dans l'URL

```
<?
print("<p>Bonjour $nom, ");
print("d'après l'URL vous avez $age ans.\n");
?>
```

Ce qui donne à l'écran : Bonjour Paul, d'après l'URL vous avez 18 ans.

Exemple 2 un peu plus poussé vous permettant de Récupérer des informations afin de créer des liens dynamiques.

Nom Script : "info.html" //un simple formulaire en HTML

```
<html>
<head>
<title>Récupérer des informations pour créer des liens
dynamiques</title>
</head>

<body>
<h2>Renseigner les champs ci dessous</h2>
// envoi les résultats du formulaires vers
"generer_liens.php3"
<form action="generer_liens.php3" method="GET">
Votre Email : <input type="text" name="email">
<br>Nom : <input type="text" name="nom">
<br>Prenom : <input type="text" name="prenom">
<p><input type="submit" name="soumettre"
value="Soumettre !">
<input type="Reset">
</form>
</body>
</html>
```

Ce banal formulaire comporte des champs de saisie textuels grâce auxquels l'utilisateur peut envoyer des informations aux prochains scripts. Les noms des champs seront les noms de variables dans le prochain script.

Nom Script : "generer_liens.php3"

```
<html>
<head>
<title>Liens générés</title>
</head>

<body>
<h2>Lien généré par la saisie de l'utilisateur</h2>
/*les liens vont être générés grâce aux informations
contenues dans l'URL après que l'utilisateur ai appuyé
sur Soumettre*/
<?
print("<p><a href='utilisateur.php3?
email=$email&nom=$nom&prenom=$prenom'>");
print("Voir ce que vous avez saisi</a>");
?>
</body>
</html>
```

Ce script affiche une page avec un lien de la forme : ? nom=\$valeur, lors du click sur ce lien, l'utilisateur pourra revoir les données qu'il à entrées précédemment ! Voyons le script qui permet de réafficher les valeurs saisies à partir de l'URL :

Nom Script : "utilisateur.php3"

```
<html>
<head>
<title>Données sur l'utilisateur provenant des liens
générés</title>
</head>
<body>
Prénom : <b><? echo $prenom ?></b>
<br>Nom : <b><? echo $nom ?></b>
<br>Email : <b><? echo $email ?></b>
</body>
</html>
```

4. UTILISATION DES COOKIES

Présentation des cookies

PHP permet de gérer très facilement les cookies, c'est-à-dire la création de fichiers texte chez le visiteur permettant de mémoriser des informations (variables) pour chaque utilisateur. Le cookie a comme particularité de gérer les informations du côté client et non serveur. Et celles-ci pourront rester plusieurs années sur le PC de votre client, ou juste quelques minutes ou encore s'effacer après avoir quitter votre site.

Un cookie est un fichier contenant des informations stocké sur le poste d'un client (celui ou s'exécute le navigateur web). Le client peut décider de ne pas accepter les cookies ... Dans ce cas, le script PHP ne pourra alors pas déposer de cookies (ou créer une session sur le poste du client). Le nombre de cookies est limité à 20 par site. Les cookies sont très utilisés car ils permettent de "conserver en mémoire" plusieurs paramètres qui pourront être accessibles sur toutes les pages d'un site.

Ecriture ou création d'un cookie

int setcookie (string name [, string value [, int expire [, string path [, string domain [, int secure]]]])

- name : nom du cookie à déposer - seul paramètre obligatoire
- value : valeur du cookie
- expire : date d'expiration du cookie
- path : restreint l'utilisation du cookie à un sous répertoire
- domain : nom de domaine du cookie (ie : le cookie ne peut être lu que de ce domain)
- secure : spécifie si le cookie doit être transmis à travers une connexion https

Note : Si seul le nom du cookie (la valeur name) est spécifié, alors le cookie correspondant est détruit sur le poste du client.

Exemple

```
<?
setcookie("moncookie","valeur du cookie",time()
+86400); // expire dans 24 heures
?>
```

Ce cookie possède 3 informations importantes: son nom (NOM_DU_COOKIE), sa valeur (VALEUR_DU_COOKIE) et la fonction time() qui définit la durée de validité du cookie (dans l'exemple, 84600 secondes soit 24 heures). Une fois cette période passée, ce fichier sera automatiquement détruit. C'est vous qui définissez le nom du cookie et la valeur qu'il porte. Sachez qu'un cookie peut être utile dans un script d'authentification de membres.

Lecture d'un cookie

La lecture du cookie s'effectue exactement de la même manière qu'un paramètre transmis par l'url ...

```
<? echo $HTTP_COOKIE_VARS["moncookie"]; ?>
ou
<? echo $_COOKIE["moncookie"]; ?>
```

Pour lire un cookie, il faut déjà savoir si le cookie a bien été défini. Dans l'exemple ci dessous que vous pouvez mettre dans n'importe quelle page de votre site, nous vérifions via l'instruction **isset()** si une donnée existe via un cookie et si oui, nous l'affichons.

```
<?
$NOM_DU_COOKIE=$_COOKIE["moncookie"];
if(isset($NOM_DU_COOKIE))
{
// Le cookie existe, on affiche les infos contenues
print "$VALEUR_DU_COOKIE";
}
else
{
print "Aucune variable transmise";
}
?>
```

Comment supprimer un cookie?

Pour supprimer un cookie, il vous suffit de mettre dans vos pages le code PHP ci dessous:

```
<?
setcookie("NOM_DU_COOKIE");
?>
```

Comme vous le voyez, il suffit d'appeler le cookie par son nom et cela le supprime.

Ecriture et lecture d'un cookie sous forme de tableau d'éléments

Ecrire un cookie sous forme de tableau d'éléments :

```
<?
setcookie( "moncookie[index1]", "1" );
```

```
setcookie( "moncookie[index2]", "2" );
setcookie( "moncookie[index3]", "3" );
?>
```

Lire un cookie contenant un tableau d'éléments :

```
<?
// si le cookie existe
if ( isset( $moncookie ) ) {

// Tant qu'il existe un élément dans le cookie faire
while( list( $nom, $valeur ) = each( $moncookie ) ) {

    // afficher la paire "nom" , "valeur"
    echo "$nom == $valeur<br>\n";
}
}
?>
```

5. REDIRIGER SUR UNE AUTRE PAGE (REDIRECTION)

IMPORTANT : Les en-têtes HTTP

Les redirections sont des en-têtes HTTP. Or, selon le protocole HTTP, les en-têtes HTTP doivent être envoyés avant tout autre type de contenu, ce qui signifie qu'aucun caractère ne doit être envoyé avant l'appel de la fonction header, pas même un espace !

En d'autre termes, **la fonction header() doit impérativement être utilisée avant tout code HTML.**

```
<html>
<?php
/* Ceci produira une erreur. Notez la sortie ci-dessus,
 * qui se trouve avant l'appel à la fonction header() */
header('Location: http://www.example.com/');
?>

<?php
header("Location: http://www.example.com/"); /* Redirec
tion du navigateur */

/* Assurez-vous que la suite du code ne soit pas exécuté
e une fois la redirection effectuée. */
exit;
?>
```

Redirection simple

Pour rediriger le visiteur vers une autre page

(particulièrement utile dans une boucle conditionnelle), il suffit d'utiliser le code suivant :

```
<?php
header('Location: mapage.php');
?>
```

Où *mapage.php* représente l'adresse de la page vers laquelle vous voulez rediriger. Cette adresse peut être absolue et peut également posséder des paramètres de la forme **mapage.php?param1=val1¶m2=val2**.

Chemin relatif / absolu

Dans l'idéal, préférez un chemin absolu à partir de la racine du serveur (DOCUMENT_ROOT), de la forme suivante :

```
<?php
header('Location: /repertoire/mapage.php');
?>
```

Si jamais la page cible est sur un autre serveur, vous mentionner l'URL complète, de la forme suivante :

```
<?php
header('Location:
http://www.commentcamarche.net/forum/');
?>
```

6. DATE ET HEURE

*Afficher la date et l'heure (classique)

Pour afficher la date et l'heure de façon classique sur votre site, il vous suffit de mettre de petit code sur votre page web:

```
<?
$date = date("d-m-Y"); $heure = date("H:i");
Print("Nous sommes le $date et il est $heure"); ?>
```

Ce qui donnera sur votre site par exemple ceci:

Nous sommes le 2-04-2002 et il est 16: 45

*Toutes les fonctions de date et heure

Dans l'exemple du dessous, nous avons mis la date au format classique (jour/mois/année)

et l'heure aussi (16 heures 45 et non pas 4H45 pm).

Vous pouvez si vous le voulez changer la façon d'afficher la date et l'heure. Voici l'ensemble des fonctions:

A - "AM" (matin) ou "PM" (après-midi)

d - Jour du mois, sur deux chiffres (éventuellement avec un zéro) : "01" à "31"

D - Jour de la semaine, en trois lettres (et en anglais) : par exemple "Fri" (pour Vendredi)

F - Mois, textuel, version longue; en anglais, i.e. "January" (pour Janvier)

h - Heure, au format 12h, "01" à "12"

H - heure, au format 24h, "00" à "23"

g - Heure, au format 12h sans les zéros initiaux, "1" à "12"

G - Heure, au format 24h sans les zéros initiaux, "0" à "23"

i - Minutes; "00" à "59"

j - Jour du mois sans les zéros initiaux: "1" à "31"

l - ('L' minuscule) - Jour de la semaine, textuel, version longue; en anglais, i.e. "Friday" (pour Vendredi)

L - Booléen pour savoir si l'année est bissextile ("1") ou pas ("0")

m - Mois; i.e. "01" à "12"

n - Mois sans les zéros initiaux; i.e. "1" à "12"

M - Mois, en trois lettres (et en anglais) : par exemple "Jan" (pour Janvier)

s - Secondes; i.e. "00" à "59"

S - Suffixe ordinal d'un nombre, en anglais, sur deux lettres: i.e. "th", "nd"

t - Nombre de jour dans le mois donné, i.e. "28" à "31"

U - Secondes depuis une époque

w - Jour de la semaine, numérique, i.e. "0" (Dimanche) to "6" (Samedi)

Y - Année, 4 chiffres; i.e. "1999"
y - Année, 2 chiffres; i.e. "99"
z - Jour de l'année; i.e. "0" à "365"
Z - Décalage horaire en secondes (i.e. "-43200" à "43200")

Pour utiliser ces fonctions, il vous suffit de remplacer le d-m-y et le H:i de l'exemple du début et de mettre les lettres que vous voulez pour changer la forme de votre date et de l'heure

7 LES VARIABLES D'ENVIRONNEMENT

Grace à PHP, vous pouvez facilement accéder a des variables qui sont en rapport avec la session utilisateur, les données propres au serveur ou des données propres à la requête.

L'exemple suivant montre comment accéder à certaines variables propres à l'utilisateur :

- Votre Navigateur \$HTTP_USER_AGENT
- Votre adresse IP \$REMOTE_ADDR
- Votre langue \$HTTP_ACCEPT_LANGUAGE
- Les types de logiciels que vous utilisez \$HTTP_ACCEPT (On peut pas exemple voir si vous utilisez word ou excel)
- Les compressions que vous acceptez \$HTTP_ACCEPT_ENCODING

Source de l'exemple	Résultat Direct
<pre><?php echo \$HTTP_USER_AGENT."
"; echo \$REMOTE_ADDR."
"; echo \$HTTP_ACCEPT_LANGUAGE. "
"; echo \$HTTP_ACCEPT."
"; echo \$HTTP_ACCEPT ?></pre>	<pre>Mozilla/4.0 (compatible; MSIE 5.01; Windows 98) 62.173.237.253 fr image/gif, image/x- bitmap, image/jpeg, image/pjpeg, application/vnd.ms- powerpoint, application/vnd.ms- excel, application/msword, application/x- shockwave-flash, */* gzip, deflate</pre>

Ensuite vous allez voir les variables du server entre autre :

- Les logiciels utilisés par le serveur Web \$SERVER_SOFTWARE
- L'adresse mail de l'administrateur du serveur \$SERVER_ADMIN
- Le nom du serveur \$SERVER_NAME (Attention ce nom peut être différent du nom de domaine si vous êtes chez un hébergeur)

Source de l'exemple	Résultat Direct
<pre><?php echo \$SERVER_SOFTWARE."
"; echo \$SERVER_ADMIN."
";</pre>	<pre>Apache manu@manucorp.co m www.manucorp.com</pre>

<pre>echo \$SERVER_NAME."
";?></pre>	
--	--

Pour les variables relatives à la requête que vous venez d'effectuer :

- La requête que vous avez effectué \$REQUEST_URI
- La partie variable de la requête \$QUERY_STRING
- l'ID unique de la requête \$UNIQUE_ID

Source de l'exemple	Résultat Direct
<pre><?php echo \$REQUEST_URI."
"; echo \$QUERY_STRING."
"; echo \$UNIQUE_ID."
"; ?></pre>	<pre>/coursphp/lecon6 file=lecon6 QS7rY8OMjqYAAAa7 WkM</pre>

Voici les autres variables d'environnement d'apache :

\$DOCUMENT_ROOT Racine du serveur
\$HTTP_ACCEPT_LANGUAGE Language du navigateur client (ex : fr)
\$HTTP_HOST Hôte du client
\$HTTP_REFERER Adresse de la page qui a conduit le client à la page courante
\$SCRIPT_FILENAME Nom du script en cours (ex : mon_script.php)
\$REQUEST_URI Adresse du script en cours (ex : http://www.site.com/script.php)
\$REQUEST_METHOD méthode utilisée pour appeler la page (GET, HEAD, POST, PUT) ;

PHP propose toute une autre série de variables prédéfinies qui sont déjà présentes dans le langage sans que vous n'ayez à les déclarer. Ces variables s'écrivent toujours en majuscules et nous fournissent divers renseignements.

Voici la liste des variables d'environnement existantes

:: Variable ::	:: Description ::
\$ _SERVER['DOCUMENT_ROOT']	Racine du serveur
\$ _SERVER['HTTP_ACCEPT_LANGUAGE']	Langage accepté par le navigateur
\$ _SERVER['HTTP_HOST']	Nom de domaine du serveur
\$ _SERVER['HTTP_USER_AGENT']	Type de navigateur
\$ _SERVER['PATH_INFO']	Chemin WEB du script
\$ _SERVER['PATH_TRANSLATED']	Chemin complet du script
\$ _SERVER['REQUEST_URI']	Chemin du script
\$ _SERVER['REMOTE_ADDR']	Adresse IP du client
\$ _SERVER['REMOTE_PORT']	Port de la requête HTTP
\$ _SERVER['QUERY_STRING']	Liste des paramètres passés au script

<code>\$_SERVER['SERVER_ADDR']</code>	Adresse IP du serveur
<code>\$_SERVER['SERVER_ADMIN']</code>	Adresse de l'administrateur du serveur
<code>\$_SERVER['SERVER_NAME']</code>	Nom local du serveur
<code>\$_SERVER['SERVER_SIGNATURE']</code>	Type de serveur
<code>\$_SERVER['REQUEST_METHOD']</code>	Méthode d'appel du script

Par exemple,

```
<?php
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
// action
} else {
// action
}
```

Bien sûr, il y a beaucoup plus de variables dépendant du serveur qui appelle le script php et vous pouvez avoir un aperçu de toutes les variables grâce à la commande **phpinfo()**

```
<?php
phpinfo();
?>
```

Variables PHP

HTTP_GET_VARS	tableau associatif pour la méthode get (nom, valeur)
HTTP_POST_VARS	tableau associatif pour la méthode post (nom, valeur)
HTTP_COOKIE_VARS	tableau associatif pour la méthode cookie (nom du cookie, contenu)
GLOBALS	tableau associatif qui contient les variables globales du script, la clé est le nom de la variable (sans le dollar) et la valeur est le contenu de la variable.

Exemple de Cookies

```
<?php
Header ("Set-Cookie: $nom=$valeur; "
. "expires=Mon, 24-Dec-2003 "
. "12:00:00 GMT");
?>
<HTML><HEAD><TITLE>Liste des cookies</TITLE>
</HEAD><BODY bgcolor='white'>
```

Liste des cookies:


```
<?php
while (list($nom, $valeur) =each($HTTP_COOKIE_VARS))
echo "<B>Nom</B> : $nom <B> " . "Valeur</B> : $valeur<BR>\n";
?>
```

8, ENVOI D'UN EMAIL

Le PHP possède une fonction géniale qui permet d'envoyer un email sans avoir à passer par un logiciel de messagerie comme le fait le HTML..

Par défaut, le langage PHP contient une fonction (mail) permettant d'envoyer très simplement un mail.

Détail de la fonction mail

mail (\$destinataires, \$sujet, \$corps, \$entete)

- **\$destinataires** est la liste des destinataires séparés de virgules
- **\$sujet** est le sujet du message
- **\$corps** est le contenu du message

\$entete est l'entête du message

Exemples d'utilisation :

Exemple 1 :Voici ci dessous un code php qui vous permet d'envoyer un email:

```
<?
// écrivez votre adresse email ci-dessous
$email="Votre_email";
$destinataire = "email_du_destinataire";
$from = "$email";
$subject ="Sujet du message";
$contentu = "Votre message ici";
@mail("$destinataire", "$subject", $contentu, "From: $email \nReply-To:$email \n");
?>
```

Voilà, c'est tout, vous envoyez un email a un destinataire qui peut vous répondre!!

Exemple 2 :

```
<?
// liste des destinataires du message
$adresse="webmaster@php-mag.com,webmaster@asp-magazine.com";

// titre du message : zone sujet
$sujet="Envoi d'un message avec la fonction Mail()";

// contenu du message
$corps="Corps du message \n si vous recevez ce message ... c'est que tout fonctionne bien ";

// Création de l'entête du message
// cette entete contient l'email de l'expéditeur ainsi que l'email pour la réponse.
$entete="From:webmaster@php-mag.com\r\nReply-To: webmaster@asp-magazine.com";
// envoi du mail
mail ($adresse,$sujet,$corps,$entete);
?>
```

Fonctions avancées

Il est possible d'utiliser la fonction Mail avec plus de précision et notamment, pouvoir envoyer un message en copie jointe, en copie jointe cachée, avec une certaine priorité, au format HTML...

Ces fonctions avancées nécessitent l'ajout, d'informations dans l'entête du message

Exemple d'ajout : Les ajouts suivants sont à insérer dans l'entête du message (\$entete)

Copie jointe : CC:webmaster@php-mag.com\n

Copie jointe cachée : BCC:test@test.com\n

Priorité de l'envoi : X-priority:3\n

Envoyer le mail au format HTML :

Content-type:text/html\n

Exemple d'envoi d'un mail au format HTML

<?

// liste des destinataires du message

\$adresse="webmaster@php-mag.com,webmaster@asp-magazine.com";

// titre du message : zone sujet

\$sujet="Envoi d'un message avec la fonction Mail()";

// contenu du message

\$corps="<html><body>Corps du message
 si vous recevez ce message ... c'est que tout fonctionne bien</body></html>";

// Création de l'entête du message

// cette entete contient l'email de l'expéditeur ainsi que l'email pour la réponse.

\$entete="Content-type:text/html\nFrom:webmaster@php-mag.com\nReply-To:webmaster@asp-magazine.com";

// envoi du mail

mail (\$adresse,\$sujet,\$corps,\$entete);
?>

9. GESTION DES FICHIERS EN PHP

Le fichier peut être ouvert dans un des modes suivants :

'r' Ouvre en lecture seule : Le pointeur est placé au début du fichier.

'r+' Ouvre en lecture/écriture : Le pointeur est placé au début du fichier.

'w' Ouvre en écriture seule : Le pointeur est placé au début du fichier. Réduit la taille du fichier à 0. Tentative de création si celui ce n'existe pas.

'w+' Ouvre en lecture/écriture : Le pointeur est placé au début du fichier. Réduit la taille du fichier à 0. Tentative de création si celui ce n'existe pas.

'a' Ouvre en écriture seule : Le pointeur est placé au début du fichier. Tentative de création si celui ce n'existe pas.

'a+' Ouvre en lecture/écriture : Le pointeur est placé à la

fin du fichier. Tentative de création si celui ce n'existe pas.

'x' Crée et ouvre le fichier en lecture seule : Le pointeur est placé au début du fichier. Si le fichier existe déjà, fopen va échouer.

'x+' Crée et ouvre le fichier en lecture/écriture : Le pointeur est placé au début du fichier. Si le fichier existe déjà, fopen va échouer.

Note

Si la fonction **fopen()** echoue lors de l'ouverture du fichier, celle ci retourne 0

Exemple :

L'exemple ci dessous génère un message si la fonction fopen() est incapable d'ouvrir le fichier spécifié.

```
<html>
<body>
<?php
if (!($f=fopen("exemple.txt","r")))
exit("Unable to open file!");
?>
</body>
</html>
```

9,1, Lecture d'un fichier ligne par ligne

Lecture d'un fichier avec file

La fonction **file** permet de **lire un fichier** ligne par ligne, cependant il est préférable de l'utiliser avec précaution sur des fichiers volumineux car elle charge tout le contenu du fichier en mémoire et peut entraîner une **erreur fatale out of memory**.

```
<?php
/*Ouvre le fichier et retourne un tableau contenant une
ligne par élément*/
$lines = file('/path/file.txt');
/*On parcourt le tableau $lines et on affiche le contenu
de chaque ligne précédée de son numéro*/

foreach ($lines as $lineNumber => $lineContent)
{
    echo $lineNumber, ' ', $lineContent;
}
?>
```

Lecture d'un fichier avec fgets

Contrairement à file, l'utilisation de **fgets** pour la **lecture d'un fichier** permet de travailler sur des gros fichiers car elle ne charge en mémoire qu'une seule ligne à la fois.

```
<?php
/*Ouverture du fichier en lecture seule*/
$handle = fopen('/path/file.txt', 'r');
/*Si on a réussi à ouvrir le fichier*/
if ($handle)
{
    /*Tant que l'on est pas à la fin du fichier*/
    while (!feof($handle))
    {
        /*On lit la ligne courante*/
        $buffer = fgets($handle);
        /*On l'affiche*/
        echo $buffer;
    }
    /*On ferme le fichier*/
    fclose($handle);
}
?>
```

*9.2. Écrire dans un fichier.

L'écriture dans un fichier texte se fait avec la fonction fwrite(). Pour faire des retour à la ligne vous devez utiliser : "\r\n"

```
<?php

$f = 'file.txt';
$text = "ma chaine de caractères";
// $handle = fopen($f, "w");
$handle = fopen($f, "a+");

// regarde si le fichier est accessible en écriture
if (is_writable($f)) {
    // Ecriture
    if (fwrite($handle, $text) === FALSE) {
        echo 'Impossible d\'écrire dans le fichier '.$f.'.';
        exit;
    }

    echo 'Ecriture terminé';

    fclose($handle);
}
else {
    echo 'Impossible d\'écrire dans le fichier '.$f.'.';
}

?>
```